MENÚ DEGUSTACIÓN DE FABRIC · POWER BI & FABRIC BARCELONA

GOBERNANZA DE DATOS EN FABRIC: LINEAGE DE DATOS CON PURVIEW

Cómo automatizamos la trazabilidad desde notebooks de Spark

SAMANTHA CRUZ | POWERED BY ISOLUTIONS | BARCELONA, JULIO 2025



HOLA, SOY SAMANTHA

SENIOR DATA ENGINEER EN ISOLUTIONS · BARCELONA

- **+8 años** en análisis de datos, pipelines y gobernanza
- **Mi stack:** Databricks, Microsoft Fabric, Azure y Purview
- **Mi foco:** arquitecturas con calidad y trazabilidad
- **Comunidad:** formaciones y charlas de ingeniería de datos e IA generativa



EN ESTA SESIÓN

Hoy: cómo automatizamos el lineage desde notebooks de Spark.





SIN LINEAGE, NO HAY CONFIANZA

¿SABEMOS DE DÓNDE VIENEN REALMENTE NUESTROS DATOS?

Desconfianza

- ¿Puedo fiarme de este número?
- ¿Cuál es la fuente original?

Auditoría

- Cumplir GDPR y normativas
- Demostrar el recorrido del dato

Tiempo perdido

- Depurar pipelines ajenos
- Ingeniería inversa constante

TAKEAWAY **Lineage: el mapa del recorrido del dato, del origen al destino.**

EL LINEAGE NATIVO SE QUEDA CORTO

LO QUE OFRECE FABRIC

- Workspace lineage a alto nivel
- Vista general del workspace
- Integrado, sin configurar nada

LO QUE FALTA

- Sin detalle entre tablas y capas
- Varios Lakehouses: mapa caótico
- Difícil seguir Bronze → Gold

TAKEAWAY **Pasamos más tiempo investigando el origen que arreglando el problema.**

¿Qué reports dependen de esta tabla? La estrategia clásica: la elimino y espero a ver quién grita primero.

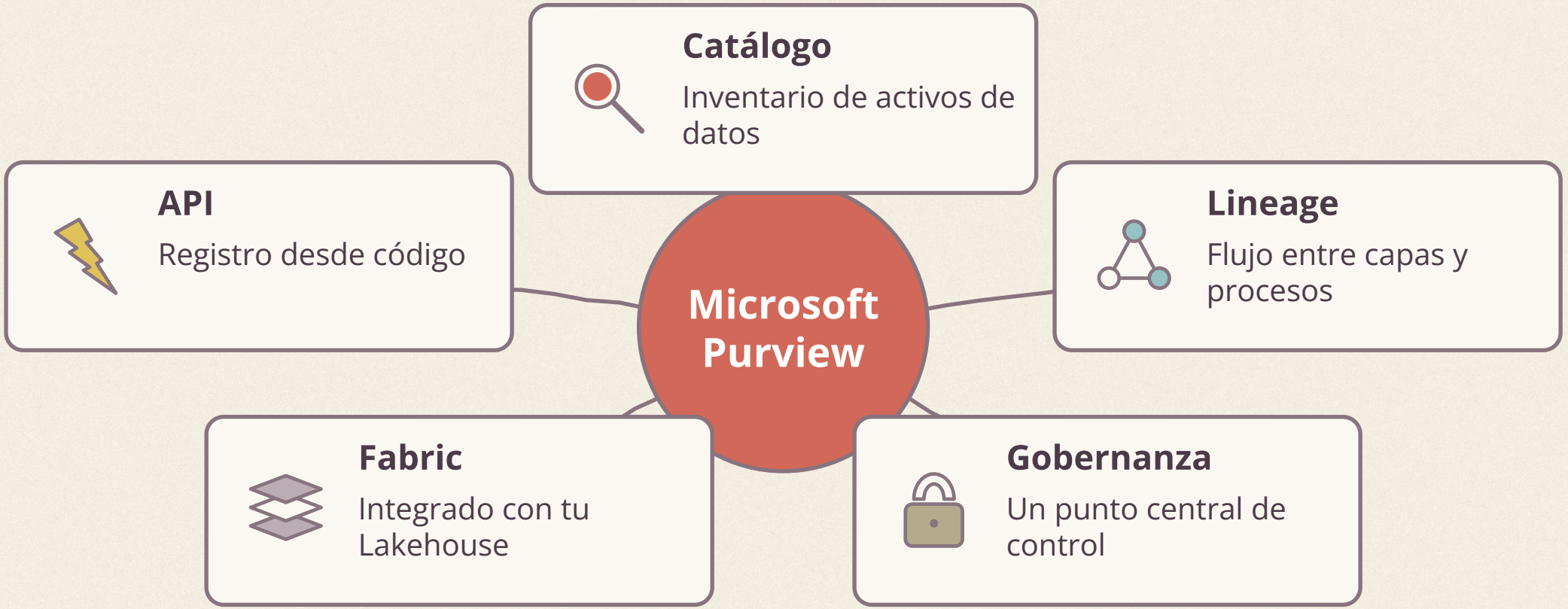
TODO INGENIERO DE DATOS, ALGUNA VEZ

sobre renombrar, mover o eliminar una tabla





PURVIEW, EL CENTRO DE GOBERNANZA



TRES SUPERPODERES DE PURVIEW

CARACTERÍSTICAS CLAVE

Descubrir

- El Google de tus datos
- Busca, encuentra y entiende
- Quién los usa y para qué

Rastrear

- Un GPS para tus datos
- De dónde viene un dato
- Qué lo transformó y a dónde va

Conectar

- SQL, Synapse, Power BI y más
- Más allá de Fabric
- Un catálogo para todo

TAKEAWAY **Gobernanza total, sin importar dónde viven tus datos.**

UN GPS PARA TUS DATOS

TRES PREGUNTAS, UNA SOLA VISTA

1 · ORIGEN

¿De dónde viene?

La fuente original del dato

1

2

2 · PROCESO

¿Qué lo transformó?

Notebooks y pipelines

3

3 · DESTINO

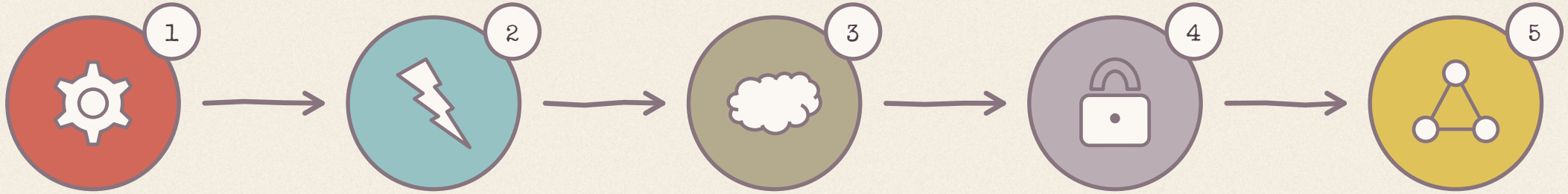
¿A dónde va?

Tablas, informes y dashboards

TAKEAWAY **Responder las tres sin hacer ingeniería inversa.**

ARQUITECTURA DE LA SOLUCIÓN

DE UN NOTEBOOK AL LINEAGE VIEW



Notebook

Spark crea la tabla y lanza la llamada

Librería

PyApacheAtlas habla con Atlas

Atlas API

Entidades y procesos en el catálogo

Seguridad

Service Principal y Key Vault

Lineage View

Activos y procesos visibles

TAKEAWAY **Un notebook, una llamada de API, lineage visible.**

PYAPACHEATLAS Y LA ATLAS API

PyApacheAtlas

- Librería Python para Atlas y Purview
- Crea, actualiza y consulta metadatos
- Clases listas: AtlasEntity, AtlasProcess

Atlas API

- REST API oficial del catálogo
- Gestiona entidades y procesos
- PyApacheAtlas la envuelve en Python

TAKEAWAY **Menos HTTP a mano, más lineage con pocas líneas.**





LA DOCUMENTACIÓN ES CLAVE

TRES ENLACES PARA EMPEZAR

RECURSO	QUÉ ENCONTRARÁS	ENLACE
PyApacheAtlas	Clases, ejemplos y requests	wjohnson.github.io/pyapacheatlas-docs
Atlas API	Referencia REST de entidades	learn.microsoft.com/rest/api/purview
Purview + Fabric	Lineage de items de Fabric	learn.microsoft.com/purview/how-to-lineage-fabric





CREDENCIALES FUERA DEL CÓDIGO

SERVICE PRINCIPAL + KEY VAULT

notebook · conexión

```
!pip install pyapacheatlas

from pyapacheatlas.auth import ServicePrincipalAuthentication
from pyapacheatlas.core import PurviewClient

KV_URL = "https://<tu-keyvault>.vault.azure.net/"
get = lambda name: mssparkutils.credentials.getSecret(KV_URL, name)

auth = ServicePrincipalAuthentication(
    tenant_id=get("tenant-id"),
    client_id=get("client-id"),
    client_secret=get("client-secret"),
)

client = PurviewClient(account_name="<tu-purview>", authentication=auth)
```

1 Key Vault

Los secretos nunca viven en el notebook.

2 Service Principal

Necesita rol en Purview para escribir.

3 PurviewClient

La puerta de entrada a la Atlas API.





UNA TABLA ES UNA ENTIDAD

SU IDENTIDAD: LA RUTA DE ONELAKE

notebook · tablas

```python
from pyapacheatlas.core.entity import AtlasEntity
from pyapacheatlas.core.util import GuidTracker

guid = GuidTracker()
ONELAKE = "https://onelake.dfs.fabric.microsoft.com"

def table_entity(name, layer, description):
    path = f"sales_lakehouse_{layer}/Tables/{layer}_schema/{name}"
    return AtlasEntity(
        name=name,
        typeName="azure_datalake_gen2_resource_set",
        qualified_name=f"{ONELAKE}/{path}",
        guid=guid.get_guid(),
        attributes={"description": description},
    )
```

1 **Entidad**

Una tabla de OneLake con su tipo.

2 **qualifiedName**

Ruta única: la identidad del activo.

3 **GuidTracker**

GUIDs temporales para enlazar el lote.





UN PROCESO ES UNA FLECHA

UNE ENTRADAS Y SALIDAS

notebook · procesos

```python
from pyapacheatlas.core import AtlasProcess

def process(name, op_type, inputs, outputs, description):
    return AtlasProcess(
        name=name, typeName="azure_synapse_operation", qualified_name=name,
        guid=guid.get_guid(), inputs=inputs, outputs=outputs,
        attributes={"operationType": op_type, "description": description},
    )

# Un notebook que lee dos tablas y escribe una
join = process(
    "notebook_join_order_details", "JOIN",
    inputs=[orders_clean, products_raw],
    outputs=[order_details],
    description="Une pedidos limpios con productos",
)
```

1 Entradas

Tablas que el proceso lee.

2 Salidas

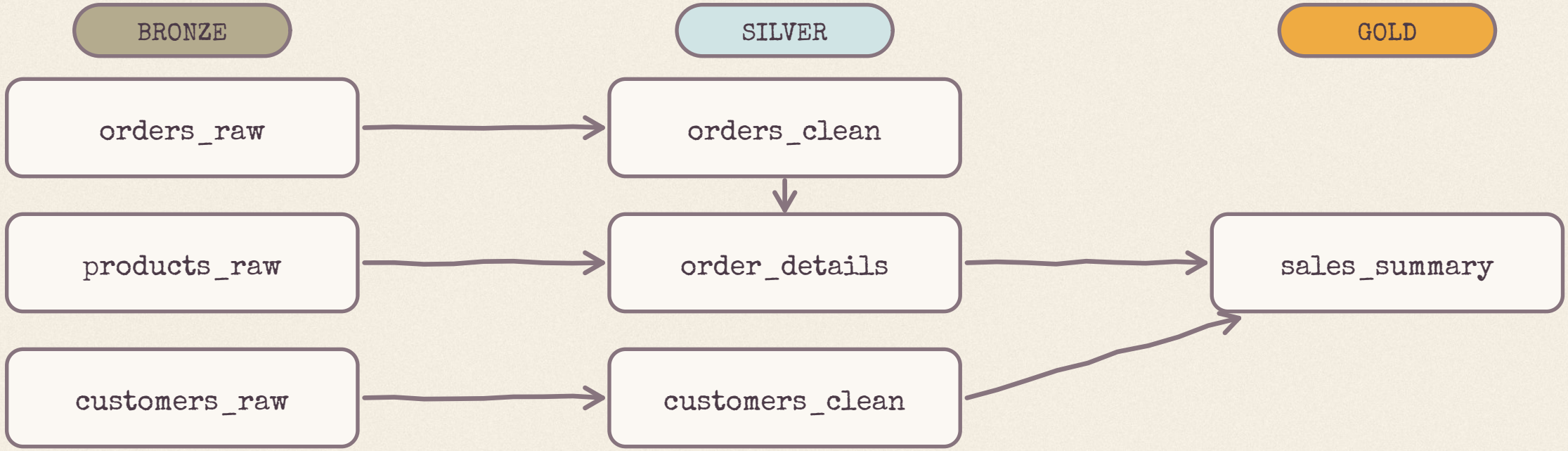
Tablas que el proceso escribe.

3 operationType

TRANSFORM, JOIN, AGGREGATE...

LO QUE VERÁS EN EL LINEAGE VIEW

BRONZE → SILVER → GOLD, TABLA A TABLA



TAKEAWAY **Cada flecha es un proceso registrado en Purview.**

LINEAGE EN POCAS LÍNEAS

TABLAS + PROCESO, UN SOLO ENVÍO

notebook · registro

```
orders_raw   = table_entity("orders_raw", "bronze", "Pedidos del ERP")
orders_clean = table_entity("orders_clean", "silver", "Pedidos limpios")

clean_orders = process(
    "notebook_clean_orders", "TRANSFORM",
    inputs=[orders_raw],
    outputs=[orders_clean],
    description="Limpia pedidos de Bronze y escribe en Silver",
)

# Un solo envío: tablas + proceso = lineage
client.upload_entities([orders_raw, orders_clean, clean_orders])
```

1 Entradas y salidas

Las tablas que lee y escribe.

2 El proceso

El notebook que las transforma.

3 upload_entities

Una llamada crea o actualiza todo.

BUSCAR Y ENRIQUECER EN PURVIEW

DEL CÓDIGO AL CATÁLOGO Y VUELTA

notebook · catálogo

```python
# Buscar activos en el catálogo
for e in client.discovery.search_entities("orders*"):
    print(e["name"], e["entityType"], e["id"])

# Añadir columnas a una tabla ya registrada
orders_raw.attributes["columns"] = [
    {"name": "order_id", "type": "string"},
    {"name": "customer_id", "type": "string"},
    {"name": "order_ts", "type": "timestamp"},
]
client.upload_entities([orders_raw])
```

1 Buscar

Comodines sobre todo el catálogo.

2 Enriquecer

Añade columnas y metadatos.

3 Actualizar

Mismo qualifiedName, misma entidad.





DEMO

DEL NOTEBOOK AL LINEAGE VIEW

¡Hora de la demo!

1. Ejecutar el notebook de lineage
2. Buscar los activos en Purview
3. Abrir el Lineage View: Bronze → Gold
4. Impacto: ¿qué rompo si borro esta tabla?





LO QUE HAY QUE SABER

ANTES DE LLEVARLO A PRODUCCIÓN

- **Registro manual:** describes cada tabla y proceso en el código
- **Mantenimiento:** si cambia el notebook, cambia el registro
- **Scan nativo:** el de Fabric llega a nivel de item, no de tabla
- **Identidad:** qualifiedName único por activo y por proceso
- **Permisos:** el Service Principal necesita rol en Purview



SIGUIENTE PASO

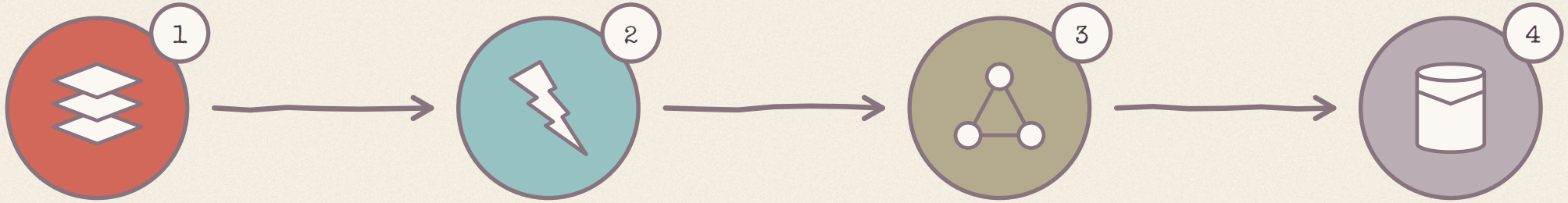
Funciona, pero lo ideal es que el lineage se registre solo.





OPENLINEAGE: LINEAGE AUTOMÁTICO

CUATRO PIEZAS PARA ACTIVARLO EN SPARK



JAR

openlineage-spark en el classpath

Listener

spark.extraListeners captura lecturas y escrituras

Endpoint

URL, transporte HTTP y namespace

Receptor

Marquez, Atlan u otro backend

TAKEAWAY **Los jobs consumen datasets y producen datasets: Spark lo registra solo.**

ACTIVAR OPENLINEAGE EN SPARK

CONFIGURACIÓN MÍNIMA

spark · openlineage

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder.appName("lineage-demo")
    .config("spark.jars.packages",
            "io.openlineage:openlineage-spark_2.12:<versión>")
    .config("spark.extraListeners",
            "io.openlineage.spark.agent.OpenLineageSparkListener")
    .config("spark.openlineage.transport.type", "http")
    .config("spark.openlineage.transport.url", "http://<marquez>:5000")
    .config("spark.openlineage.namespace", "fabric_lakehouse")
    .getOrCreate()
)
```

1 Paquete

openlineage-spark, versión fijada.

2 Listener

Spark avisa en cada lectura y escritura.

3 Destino

Endpoint HTTP y namespace.

¡GRACIAS!

Lineage = confianza, auditoría y menos depuración
Purview + Atlas API: registro automatizado desde Spark
OpenLineage: el siguiente paso hacia tiempo real

SAMMCRUZ.GITHUB.IO

